

DxxxxR0: Naming improvements for std::execution

1 Abstract

This paper proposes naming improvements for the sender factories and sender adaptors presented in P2300R4 std::execution.

2 Paper history

R0: initial revision

3 List of names with suggested changes

P2300R4 name	suggested name	alternative name	
schedule			
just	set_value	value	
transfer_just	transfer_set_value	<i>remove</i>	
just_error	set_error	error	
just_stopped	set_stopped	stopped	
read			
transfer			
then		transform	
upon_error/stopped	then_error/then_stopped	transform_error/transform_stopped	
let_value/error/stopped	?		
on			
into_variant			
stopped_as_optional			
stopped_as_error			
bulk			
split			
when_all			
transfer_when_all		<i>remove</i>	
ensure_started			

start_detached	
this_thread::sync_wait	
execute	

4 Discussion

Senders/receivers employ three channels:

- value channel
- error channel
- stopped channel

Operations that are available for all three channels should call out the specific channel they operate on, in a consistent manner. If, instead, the value channel is considered sufficiently default that it need not be named explicitly, it should never be named. Existing practice in P2300R4 appears to gravitate towards always naming the channel.

Compare `let_value` / `let_error` / `let_stopped` with `just` / `just_error` / `just_stopped` for an obvious inconsistency in this regard.

Looking at the existing names, there seems to be little consistency in the kinds of words being used: verbs (`schedule`, `transfer`, `execute`), verb-based phrases (`stopped_as_optional`), and adverbs (`just`, `then`, `when_all`) are intermixed.

For me, particularly and negatively outstanding is the rather unsophisticated `just` to indicate the injection of values in the sender chain. The proposal here is to use either `set_value` / `set_error` / `set_stopped` or plain `value` / `error` / `stopped` as the consistent triplet, with the author's slight preference towards the `set_*` names for consistency with `let_*`.

The counterparts for `then` are `upon_error` and `upon_stopped`, which is inconsistent. Also, the name `transform` is used in other areas of the standard to describe the application of a functor on some input value(s), e.g. `std::transform` or `std::ranges::view::transform`. For `then` in particular, such cross-domain consistency may not carry its weight compared to the idiomatic `then` in the local domain.

The rationale for `let_value` is the similarity to a LISP-style "let" operation that introduces a local variable. However, examples seem to favor a `fork` or `spawn` reading. I do not have sufficient experience with the actual usage of senders/receivers to form an opinion whether such examples warrant a rename of the `let_*` adaptors.

Finally, if at all possible, compound operations such as `transfer_just` or `transfer_when_all` should be left as an implementation optimization (possibly specified in the standard); however, the user should not need to learn whether a certain combination of sender adaptors has a compound spelling or not.

5 Proposal

- Rename `just` / `just_error` / `just_stopped` to `set_value` / `set_error` / `set_stopped`.
- Consider renaming `upon_error` and `upon_stopped` to `then_error` and `then_stopped`.